Protocolo Executável — Testing Skill Profile (GitSkills × Practical Test Pyramid)

MSR 2027 Mining Challenge · 16/out/2026

Objetivo e contribuições

O estudo identifica quais práticas de teste as agent skills do domínio blockchain instruem o agente a seguir, tendo a Practical Test Pyramid (Vocke, 2018) como referência. Uma LLM, validada por anotação humana, classifica cada skill quanto às camadas de teste instruídas, ao contexto (on-chain, off-chain Web3 ou Web2) e às boas e más práticas presentes, sempre com evidência textual.

- **C1 — Testing Skill Profile:** quais camadas de teste as skills blockchain instruem, por contexto.
- **C2 — Catálogo de práticas:** boas e más práticas de teste instruídas, categorizadas e contabilizadas, cada uma com citação e justificativa.
- **C3 — Instrumento e dados:** checklist, codebook, prompt e anotações, reutilizáveis em outros recortes do GitSkills.

Prazos (AoE): abstract **18/dez/2026**, paper **23/dez/2026**. 4 páginas + 1 de referências, IEEEtran 10pt, duplo-anonimizado, seção de ética obrigatória (call).

Questões de pesquisa

Dados: GitSkills, snapshot jul/2026 (Zenodo 10.5281/zenodo.21875637); unidade de análise = um `SKILL.md` distinto.

- **RQ1 — Perfil:** quais camadas de teste as skills instruem, e em que contexto (on-chain, off-chain Web3, Web2)?
- **RQ2 — Práticas:** quais boas e más práticas de teste as skills instruem, e com que frequência por categoria e contexto?

Contextos: on-chain, off-chain Web3 e Web2

O contexto é definido pelo que o componente testado faz, não pelo repositório onde está.

Contexto	Definição	Exemplos
On-chain	Código executado pela blockchain	Smart contracts (Solidity, Vyper, Rust/ Anchor, Move), proxies, scripts de deploy
Off-chain Web3	Código fora da chain que lê/escreve estado, assina transações, gerencia chaves ou consome ABI/eventos	dApp com wallet, SDK/RPC (ethers, viem), indexadores, oráculos, bots, bridges
Web2	Código sem interação com a chain, mesmo num projeto Web3	API REST, banco, autenticação, UI sem wallet, CI genérico

Cada skill recebe também uma classe de domínio: `onchain`, `offchain_web3`, `misto`, `web2_incidental` (blockchain é menção periférica) ou `indeterminado`. Isso responde se as skills do recorte são predominantemente Web3 ou Web2.

Metodologia

A metodologia segue uma abordagem MSR em seis fases. Convenções: um script por fase, seed **20270402**, execuções registradas em `RUNLOG.md`, e todo número do paper lido de um JSON gerado por script.

Fase 1 — Seleção das skills

- **Recorte de domínio:** termos blockchain gerados por LLM e revisados por dois pesquisadores, separados em fortes (bastam sozinhos: solidity, smart contract, foundry, hardhat...) e fracos (só com ≥ 2 ou junto de um forte: wallet, token, gas...). Aplicados a nome, descrição e conteúdo das skills com `dedup_primary = 1`.
- **Filtro de teste:** frases compostas de teste (ex.: "unit test", "forge test", "fuzz"), geradas e revisadas da mesma forma. O filtro só seleciona candidatas; a classificação é feita nas fases 4 e 5.
- **Validação da seleção:** um pesquisador verifica 100 skills sorteadas do recorte ("é blockchain/Web3?") e 100 do pool ("fala de teste?"). Reportar a precisão com IC 95%; meta ≥ 0,90.
- **Saída:** pool de candidatas (≈ 17,8 mil), com número de cópias e de repositórios por conteúdo.

Fase 2 — Instrumento

O mesmo texto serve de guideline humano e de prompt. Pergunta única: o texto **instrui o agente** a fazer isto? Mencionar ou listar não conta.

Checklist de camadas. O critério de YES parafraseia a seção correspondente de Vocke; a adaptação a Web3 segue a documentação do Foundry, Hardhat e Anchor.

Item	YES quando instrui...	Em blockchain (on / off-chain)	Não conta
Q1 Unitário	unidade isolada, doubles para colaboradores	função do contrato com dependências mockadas / função de SDK com RPC stubado	"run the tests"
Q2 Integração	junção real com parte externa	contrato ↔ contrato em chain local / backend ↔ nó RPC, eventos	fork sem verificar a junção
Q3 Contrato	consumidor e provedor respeitam interface	contrato cumpre ABI/ERC / front e indexador conferem ABI e eventos	**teste do smart contract em si**
Q4 UI	entrada → ação → estado (GUI, REST, CLI)	chamada via `cast` / dApp com wallet	XSS, injeção
Q5 E2E	sistema inteiro por interface externa	usuário → front → wallet → contrato → indexador	mesma citação de Q2
Q6 Aceitação	cenário de usuário (given/when/then)	regra de negócio do contrato / jornada no dApp	"all tests must pass"
Q7 Exploratório	exploração manual explícita	sessão em testnet / no dApp	texto informal
Q8 Automação	runner nomeado ou CI	`forge test`, `hardhat test` / `jest`, Playwright	"verification commands"
Q9 Web3	fork, fuzz, invariant, formal, gas	quase sempre on-chain	—

Codebook de práticas. Categorias e polaridade vêm de fontes: Vocke, *test smells* (Garousi & Küçük, 2018), Foundry Book, ConsenSys Smart Contract Best Practices e OWASP Smart Contract Top 10.

Categoria	Boa prática (exemplo)	Má prática (exemplo)
P1 Equilíbrio da pirâmide	muitos unitários, poucos E2E	só E2E ou só teste manual

Categoria	Boa prática (exemplo)	Má prática (exemplo)
P2 Isolamento	mockar oráculo e contratos externos	unitário dependente de RPC público
P3 Asserções	revert com erro exato, eventos, limites	só caminho feliz; teste sem asserção
P4 Determinismo	fork com bloco fixo, tempo controlado	fork sem bloco fixo; testnet instável
P5 Adversarial	fuzz/invariant, reentrância, acesso	só exemplos fixos em código de valor
P6 Automação e CI	testes no CI bloqueando merge	execução só manual
P7 Cobertura	cobertura como sinal	% de cobertura como objetivo
P8 Integridade	corrigir o código, não o teste	pular ou afrouxar teste que falha
P9 Segredos	contas locais, segredos fora do repo	chave privada real no teste
P10 Outras	surgidas na calibração	surgidas na calibração

Regras de evidência (humanos e LLM)

- Cada YES de camada e cada prática exige **citação literal** (5–40 palavras) e uma **justificativa** de uma frase que liga a citação ao critério ou à categoria.
- Cada evidência recebe um contexto: on-chain, off-chain Web3 ou Web2.
- Só conta o que é **instruído**; a polaridade segue o codebook, não a opinião do anotador.
- Uma citação não pode justificar dois itens diferentes.

Calibração. Dois pesquisadores e a LLM anotam 30 skills fora da amostra; ajustam os exemplos uma vez (incluindo novas categorias em P10) e congelam o instrumento.

Fase 3 — Amostragem

- **Amostra principal:** 384 skills sorteadas do pool (95% de confiança, margem ± 5%), no máximo 1 por repositório e por grupo de quase-duplicatas (MinHash, Jaccard ≥ 0,8).
- **Subamostra humana:** 60 skills sorteadas dentro das 384.
- **Texto exibido:** nome + descrição + conteúdo até 8.000 caracteres, idêntico para LLM e humanos.

Fase 4 — Classificação por LLM

- **Saída por skill (JSON):** classe de domínio; Q1–Q9 com resposta, citação, justificativa e contexto; lista de **boas e más práticas**, cada uma com citação, categoria, polaridade, contexto e justificativa.
- **Checagem automática:** a citação existe no texto e não se repete entre itens. Se falhar duas vezes, o item vira UNCERTAIN e é contado como descartado.
- **Configuração:** modelo e versão exatos, temperatura 0, prompt com hash; 3 execuções nas 60 da subamostra para medir estabilidade.

Fase 5 — Anotação humana e validação

- **Anotação cega:** R1 e R2 anotam as 60 skills de forma independente, sem ver a saída da LLM (≈ 10 min por skill).
- **Concordância humana:** Cohen κ por item (Gwet AC1 nos itens raros); depois, adjudicação → consenso.
- **LLM × consenso:** κ e F1 por camada, por contexto e por classe de domínio; nas práticas, precisão e recall (citações com sobreposição ≥ 50%) e κ de categoria e polaridade.
- **Regra de corte:** item em que a LLM tem κ < 0,40 é reportado só com o valor humano.

Fase 6 — Análise e paper

- **RQ1:** % de skills com YES por camada (IC 95%), em barras empilhadas por contexto, e a composição Web3 × Web2.
- **RQ2:** contagem de boas e más práticas por categoria e contexto, com um exemplo curto de cada polaridade.
- **Exibições:** Fig. 1 (perfil por contexto), Tab. 1 (catálogo de práticas), Tab. 2 (dados e concordância).
- **Seções:** Introdução · Método · Resultados · Discussão · Ameaças · **Ética** · Conclusão e disponibilidade dos dados (repositório anonimizado).

Cronograma

Semana	Entrega
19–30/out	Fase 1
02–06/nov	Fase 2 e calibração
09–27/nov	Fases 3, 4 e 5
30/nov–04/dez	Concordância e adjudicação
07–11/dez	Fase 6 e rascunho; cadastrar abstract no HotCRP
18/dez	Abstract final
23/dez	Paper

Ameaças à validade

- **Seleção por termos:** precisão do recorte e do filtro medida por amostragem (Fase 1).
- **LLM superclassifica:** consenso humano como referência, citação obrigatória e checagem automática.
- **Polaridade subjetiva:** ancorada em fontes no codebook, com concordância medida.
- **Fronteira on/off-chain/Web2:** regra de decisão explícita e κ reportado.
- **Instrução ≠ prática real:** conclusões restritas ao que a skill instrui.
- **Cópias** (50,5% do GitSkills são verbatim): dedup e 1 skill por repositório.

Evolução para DevOps: instrução × prática no CI (TCC)

Objetivo: verificar se as práticas de teste instruídas nas skills são executadas no pipeline de CI dos mesmos repositórios (*instruction–practice gap*). A metodologia segue uma abordagem MSR em quatro fases e reaproveita a amostra, os IDs e os rótulos do estudo principal. O trabalho começa em janeiro, após a submissão ao MSR; até lá, o aluno atua como revisor R2 na Fase 5.

Fase 1 — Coleta dos workflows de CI

- **Fonte de dados:** repositórios das 384 skills classificadas na Fase 4. Usamos GitHub Actions porque é o CI nativo do GitHub e seus arquivos ficam versionados junto ao código.
- **Coleta:** script com token da API REST do GitHub. Para cada repositório, localizar o último commit até a data do snapshot do GitSkills (`/commits?until=`) e baixar os arquivos de `.github/workflows/` nesse commit.
- **Critério de seleção:** manter repositórios acessíveis, com ≥ 1 workflow e ≥ 1 execução registrada (`/actions/runs`). Registrar quantos saíram e o motivo. Outros CIs (GitLab, CircleCI) ficam fora do escopo e entram como limitação.

- **Saída:** `ci_workflows.jsonl` (repositório, arquivo, commit, conteúdo).

Fase 2 — Extração dos estágios de teste

- **Parsing:** script que lê cada YAML e extrai jobs, gatilhos (`on:`), comandos (`run:`) e actions (`uses:`).
- **Dicionário de comandos:** gerado por LLM e validado por dois revisores, com o mesmo procedimento de revisão de termos da Fase 1. Ele mapeia cada comando para uma camada ou prática (ex.: `forge test` → automação on-chain; `--fork-url` → fork; `forge coverage` → cobertura; `slither` → análise estática; `playwright test` → UI/E2E off-chain).
- **Contexto:** cada estágio recebe on-chain, off-chain Web3 ou Web2, pela mesma regra de decisão do protocolo.
- **Quality gate:** marcar se o estágio roda em `pull_request` e se a falha interrompe o job (sem `continue-on-error`). A proteção de branch não é visível pela API e entra como limitação.

Fase 3 — Comparação e validação

- **Cruzamento:** para cada skill, comparar os itens com YES (Q1–Q9 e práticas P1–P10) com os estágios encontrados no CI do mesmo repositório.
- **Classificação:** cada item vira *instruído e executado*, *instruído e não executado* ou *executado e não instruído*.
- **Validação humana:** dois revisores conferem 50 repositórios sorteados (skill + YAML) e indicam se concordam com o cruzamento automático. Reportar concordância bruta e κ; divergências resolvidas por consenso.

Fase 4 — Análise e entregáveis

- **Métricas:** taxa de execução por camada e por contexto; cobertura on-chain × off-chain no CI; frequência de quality gates.
- **Figura principal:** barras por camada, instruído × executado.
- **Entregáveis:** dataset de workflows, dicionário de comandos validado, scripts (mesmas convenções do protocolo), texto do TCC e um artigo derivado (ex.: SAST ou SBES).

Outras evoluções possíveis

- Aplicar checklist e codebook a skills de DevOps (deploy, CI, infraestrutura).
- Usar o histórico de commits para ver se a adoção de uma skill de teste muda o CI.
- Experimento com agente, com e sem skill de teste, medindo o efeito no CI.
- Skills que instruem o agente a executar comandos, acessar a rede ou usar segredos no CI (supply chain).

Fontes

- MSR 2027 Mining Challenge
- Destefanis et al., GitSkills, arXiv:2608.10906
- Vocke, The Practical Test Pyramid (2018)
- Garousi & Küçük, Smells in software test code: a survey of knowledge in industry and academia, JSS (2018)
- Foundry Book; ConsenSys Smart Contract Best Practices; OWASP Smart Contract Top 10